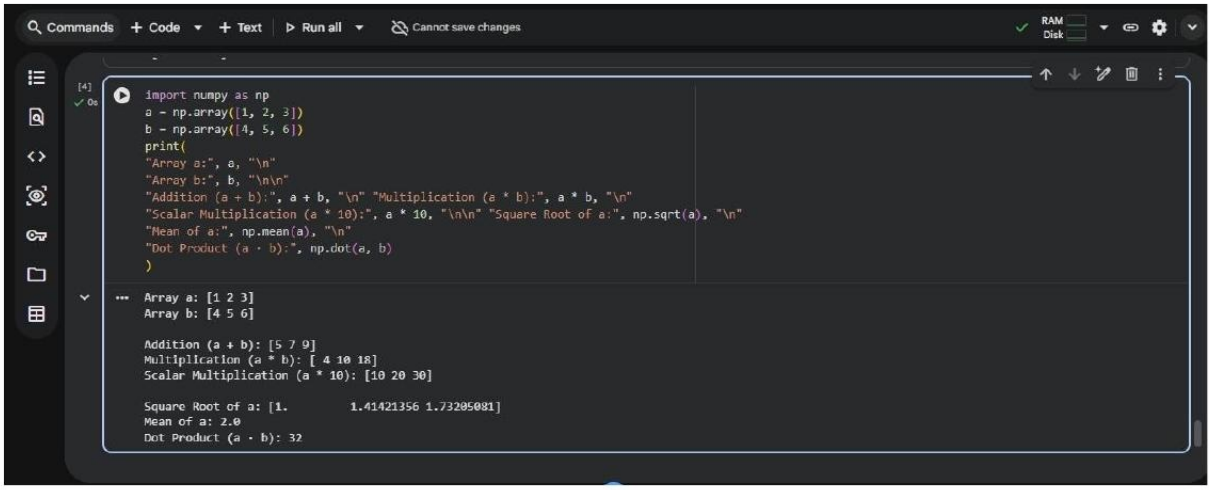
A. Creating Arrays

Aim

To write a Python program using the NumPy library to create and display different types of arrays, such as 1D arrays, 2D arrays, zeros, ones, identity matrices, and range of numbers.

Algorithm

1. Start the program.
2. Import the NumPy library as np.
3. Create a 1D array using np.array().
4. Create a 2D array using np.array().
5. Create a zero matrix using np.zeros().
6. Create a matrix of ones using np.ones().
7. Create an identity matrix using np.eye().
8. Generate a sequence of numbers using np.arange().
9. Display all the arrays using the print() function.
10. Stop the program.



Result

Thus, the Python program to create and display different types of NumPy arrays was successfully executed.

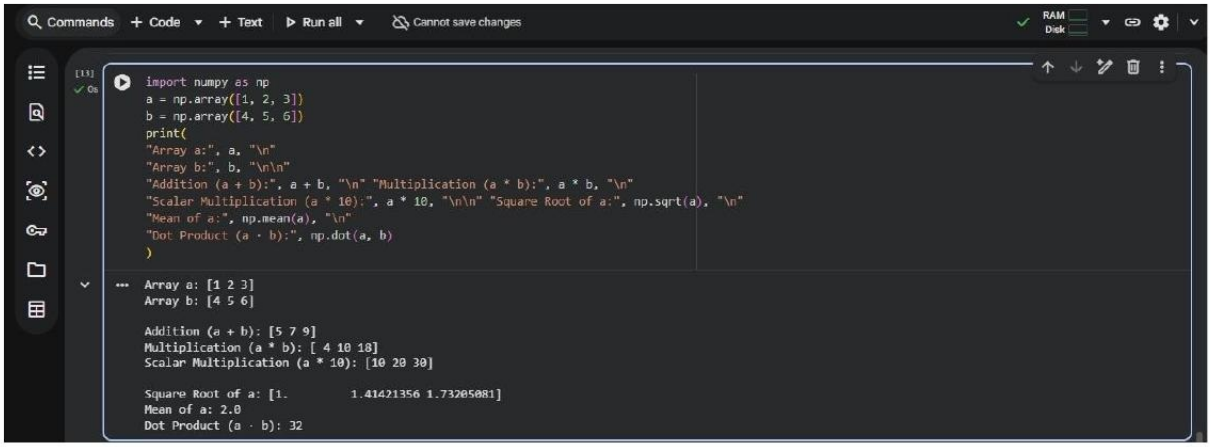
B. Array Operations

Aim

To write a Python program using NumPy to perform array operations such as addition, multiplication, scalar multiplication, square root, mean, and dot product.

Algorithm

1. Start the program.
2. Import the NumPy library as np.
3. Create two arrays, a = [1, 2, 3] and b = [4, 5, 6].
4. Display both arrays.
5. Perform element-wise addition using a + b.
6. Perform element-wise multiplication using a * b.
7. Perform scalar multiplication using a * 10.
8. Calculate the square root of array a using np.sqrt().
9. Calculate the mean of array a using np.mean().
10. Calculate the dot product of arrays a and b using np.dot().
11. Display all the results.
12. Stop the program.



Result

Thus, the Python program to perform various NumPy array operations, including addition, multiplication, scalar multiplication, square root, mean, and dot product, was successfully executed.

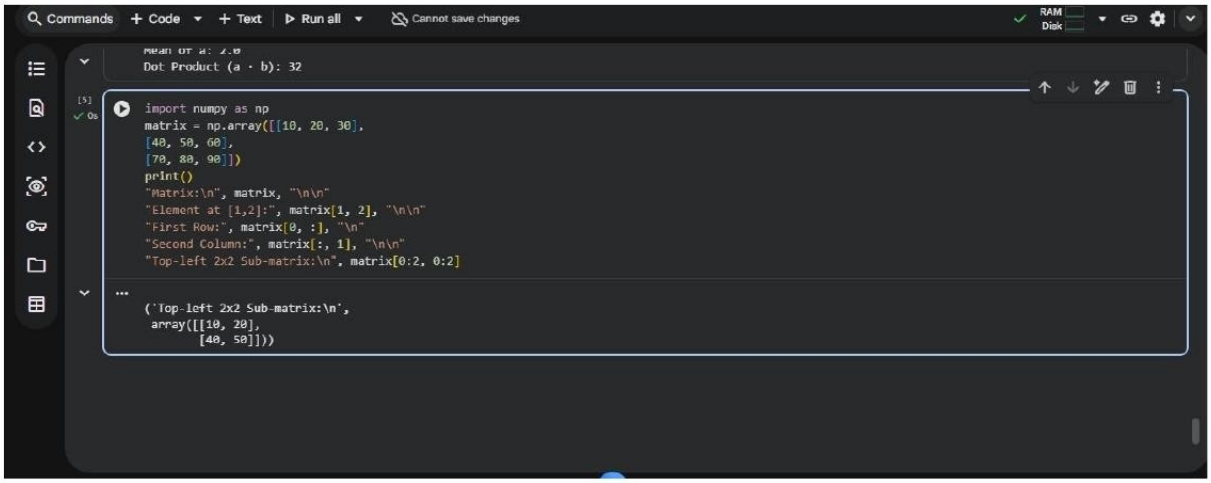
C. Indexing and Slicing

Aim

To write a Python program using NumPy to access individual elements, extract rows and columns, and obtain a sub-matrix from a two-dimensional array using indexing and slicing.

Algorithm

1. Start the program.
2. Import the NumPy library as np.
3. Create a 3×3 matrix containing the elements 10 to 90.
4. Display the matrix.
5. Access the element at index [1, 2] using matrix[1, 2].
6. Extract the first row using matrix[0, :].
7. Extract the second column using matrix[:, 1].
8. Extract the top-left 2×2 sub-matrix using matrix[0:2, 0:2].
9. Display all the extracted elements and sub-matrices.
10. Stop the program.



Result

Thus, the Python program to perform indexing and slicing operations on a NumPy matrix was successfully executed, and the required elements, row, column, and sub-matrix were obtained.

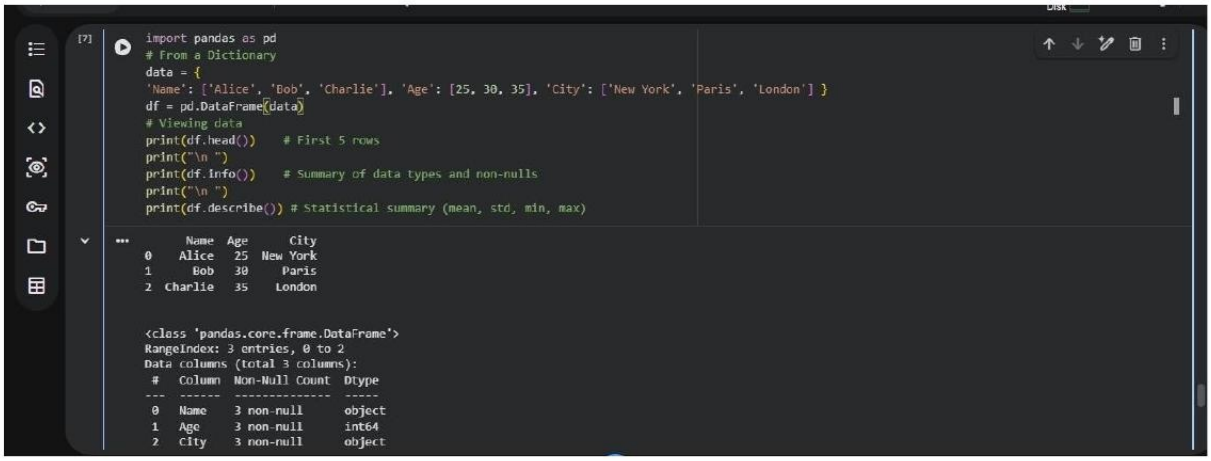
A. Creating DataFrames

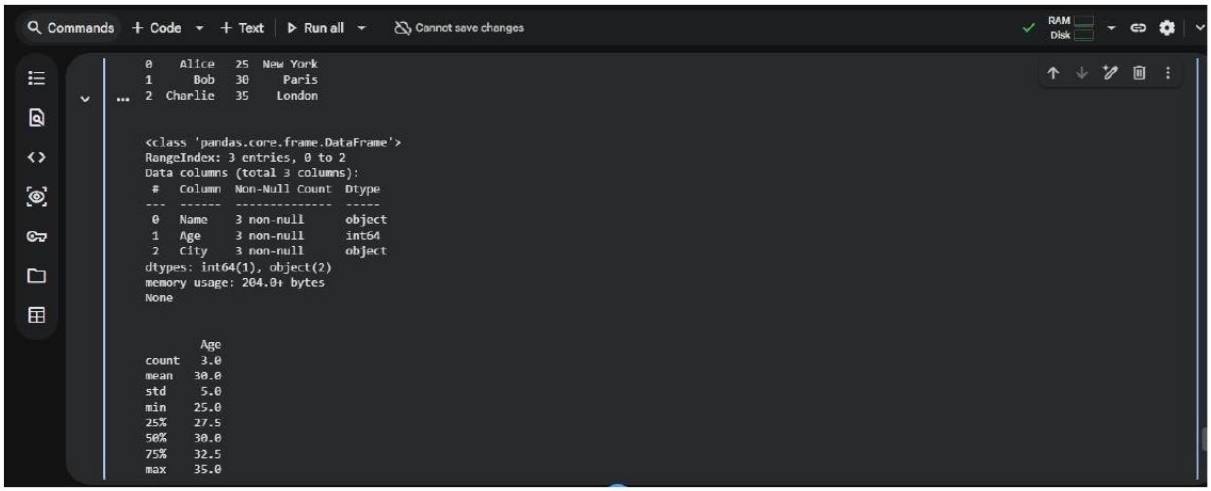
Aim

To write a Python program using the Pandas library to create a DataFrame from a dictionary and display the first few rows, data types, non-null values, and statistical summary.

Algorithm

1. Start the program.
2. Import the Pandas library as pd.
3. Create a dictionary containing Name, Age, and City.
4. Convert the dictionary into a DataFrame using pd.DataFrame().
5. Display the first five rows using df.head().
6. Display the DataFrame information using df.info().
7. Display the statistical summary using df.describe().
8. Stop the program.





Result

Thus, the Python program to create a Pandas DataFrame and display its contents, data types, and statistical summary was successfully executed.

B. Selection and Filtering

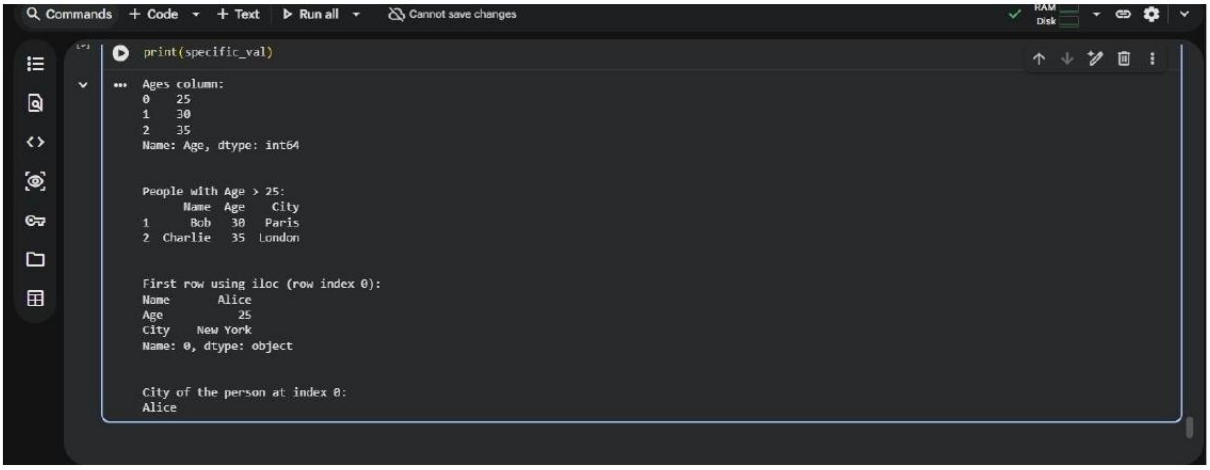
Aim

To write a Python program using Pandas to select specific columns, filter rows based on conditions, and access data using iloc and loc.

Algorithm

1. Start the program.
2. Import the Pandas library as pd.
3. Use the DataFrame created in Program A.
4. Select the Age column using df['Age'].
5. Filter rows where Age is greater than 25.
6. Access the first row using df.iloc[0].
7. Access the Name value at index 0 using df.loc[0, 'Name'].
8. Display the selected column, filtered rows, and accessed values.
9. Stop the program.





Result

Thus, the Python program to select columns, filter rows, and access DataFrame elements using iloc and loc was successfully executed.

Note: In your code, the statement df.loc[0, 'Name'] retrieves Alice, not the city. To retrieve the city, use df.loc[0, 'City'].

C. Data Cleaning (Handling Missing Values)

Aim

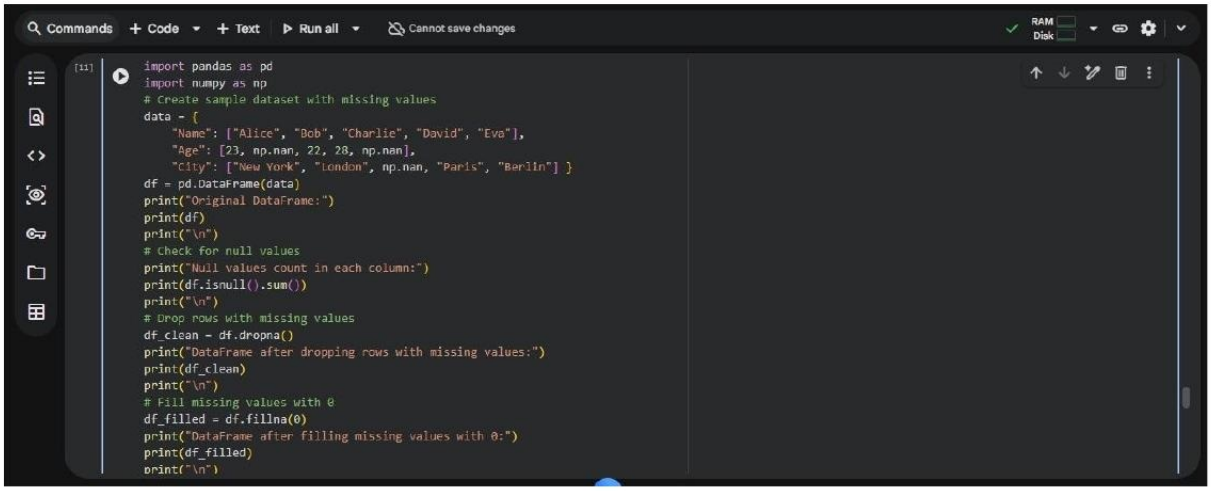
To write a Python program using Pandas and NumPy to identify and handle missing values in a DataFrame by dropping rows, filling missing values with zero, and replacing missing Age values with the column mean.

Algorithm

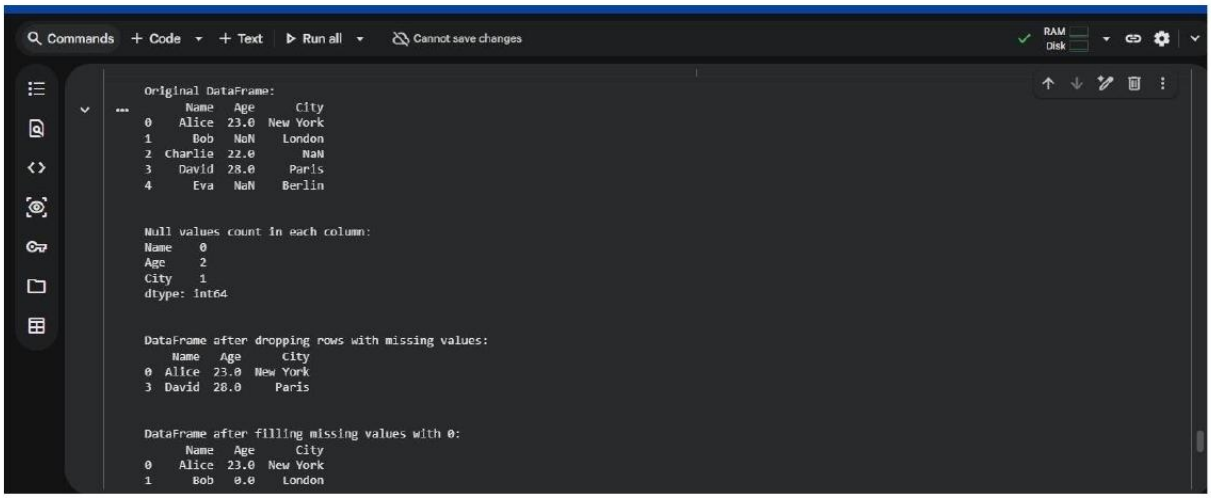
1. Start the program.
2. Import the Pandas library as pd and the NumPy library as np.
3. Create a dictionary containing Name, Age, and City, with some missing values represented by np.nan.
4. Convert the dictionary into a DataFrame using pd.DataFrame().
5. Display the original DataFrame.
6. Identify missing values in each column using df.isnull().sum().
7. Remove rows containing missing values using df.dropna().
8. Fill all missing values with zero using df.fillna(0).
9. Calculate the mean of the Age column and replace missing Age values with the mean.

10. Display the results of each cleaning operation.

11. Stop the program.



```
import pandas as pd
import numpy as np
# Create sample dataset with missing values
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva"],
    "Age": [23, np.nan, 22, 28, np.nan],
    "City": ["New York", "London", np.nan, "Paris", "Berlin"] }
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print("\n")
# Check for null values
print("Null values count in each column:")
print(df.isnull().sum())
print("\n")
# Drop rows with missing values
df_clean = df.dropna()
print("DataFrame after dropping rows with missing values:")
print(df_clean)
print("\n")
# Fill missing values with 0
df_filled = df.fillna(0)
print("DataFrame after filling missing values with 0:")
print(df_filled)
print("\n")
```



```
Original DataFrame:
      Name   Age      City
0    Alice  23.0  New York
1      Bob   NaN    London
2  Charlie  22.0       NaN
3    David  28.0     Paris
4      Eva   NaN    Berlin


Null values count in each column:
Name    0
Age     2
City    1
dtype: int64


DataFrame after dropping rows with missing values:
    Name   Age      City
0  Alice  23.0  New York
3  David  28.0     Paris


DataFrame after filling missing values with 0:
      Name   Age      City
0    Alice  23.0  New York
1      Bob   0.0    London
```

Result

Thus, the Python program to identify and handle missing values in a Pandas DataFrame using isnull(), dropna(), and fillna() was successfully executed.

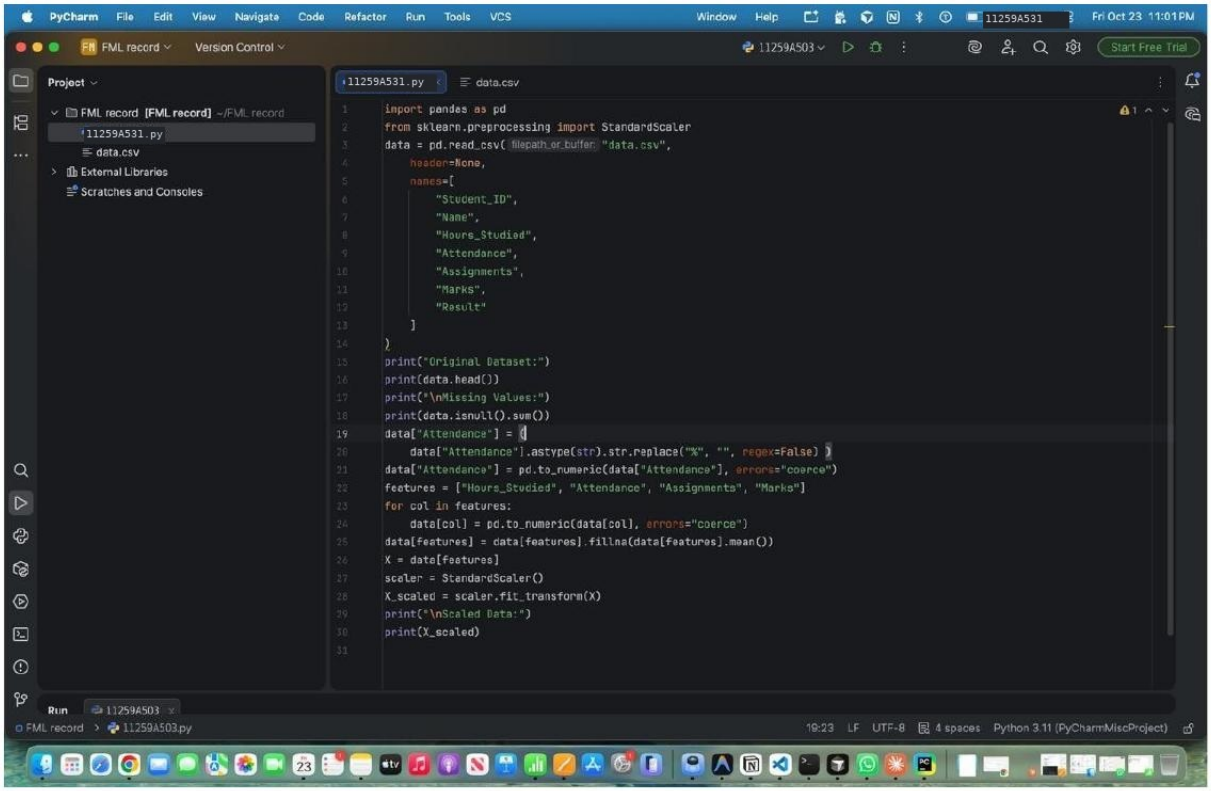
Program 1: Data Preprocessing

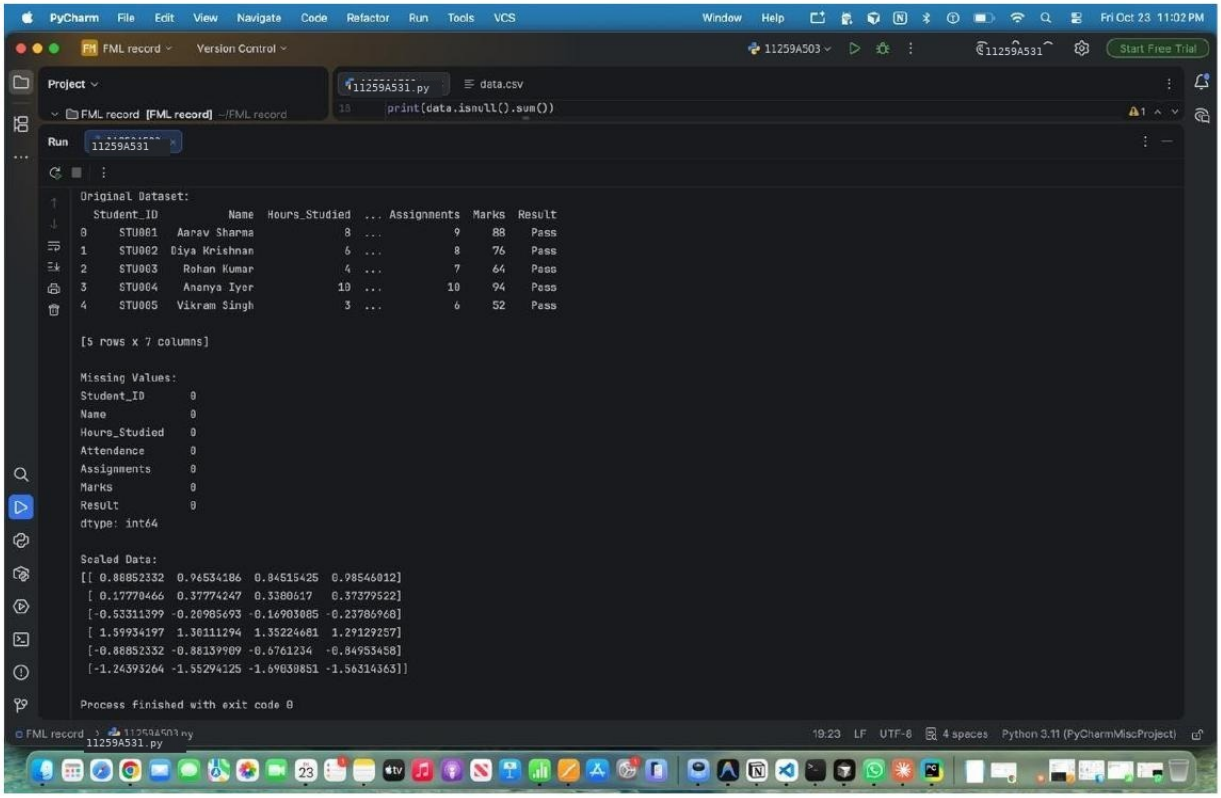
1. Aim

To preprocess a dataset by checking for missing values, filling missing numerical values with their mean, selecting numerical columns, and normalizing the data using StandardScaler.

2. Algorithm

1. Start.
2. Import the required libraries: pandas and StandardScaler from sklearn.preprocessing.
3. Load the dataset from data.csv.
4. Display the first five rows of the original dataset.
5. Check and display the number of missing values in each column.
6. Fill missing numerical values with the mean of their respective columns.
7. Select the columns Hours_Studied, Attendance, Assignments, and Marks.
8. Create a StandardScaler object.
9. Apply standardization to the selected numerical columns.
10. Convert the normalized data into a Pandas DataFrame.
11. Display the first five rows of the normalized dataset.
12. Stop.





result

Thus, the data preprocessing program was implemented successfully. It loads the dataset, identifies and handles missing numerical values, selects the required features, and standardizes

them using StandardScaler, preparing the data for further machine learning analysis.

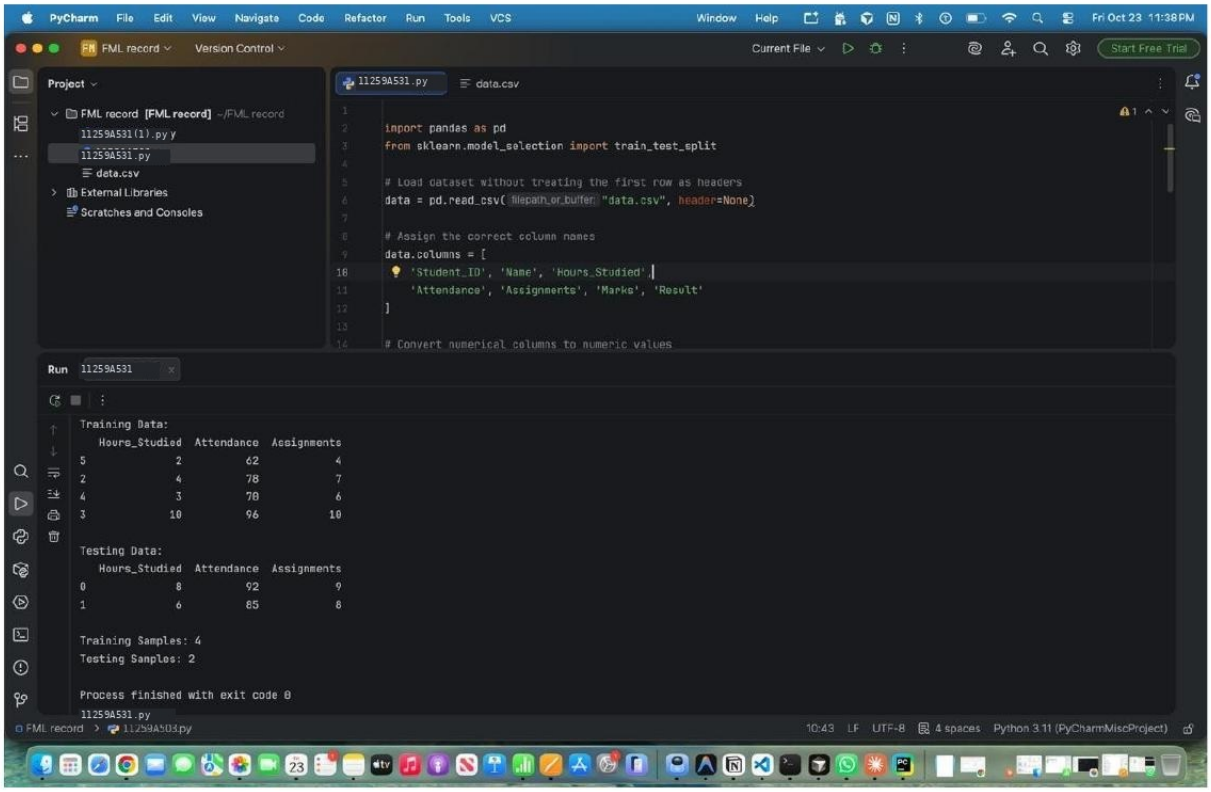
Data Splitting

Aim

To split the dataset into training and testing sets, where 80% of the data is used for training and 20% is used for testing, using the train_test_split() function.

Algorithm

1. Start.
2. Import the required libraries: pandas and train_test_split.
3. Load the dataset from data.csv.
4. Select the input features: Hours_Studied, Attendance, and Assignments.
5. Select the target variable Result.
6. Split the dataset into 80% training data and 20% testing data using train_test_split().
7. Set random_state=42 to ensure reproducible results.
8. Display the training and testing datasets.
9. Display the number of training and testing samples.
10. Stop.



Result

The dataset was successfully split into training and testing sets using an 80:20 ratio.

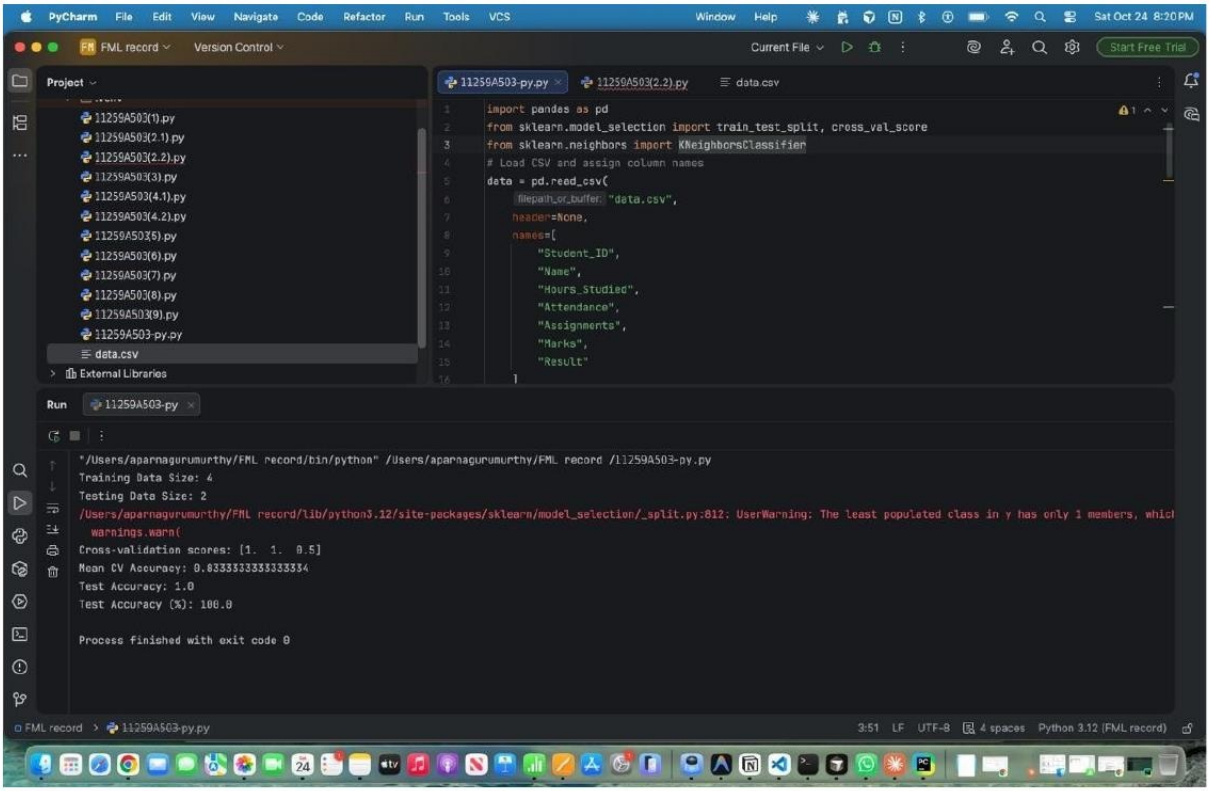
Data Split-Up and Cross-Validation

Aim

To split the dataset into training and testing sets and evaluate a K-Nearest Neighbors (KNN) classifier using 5-fold cross-validation.

Algorithm

1. Start.
2. Import pandas, train_test_split, cross_val_score, and KNeighborsClassifier.
3. Load the dataset from data.csv.
4. Select the input features: Hours_Studied, Attendance, and Assignments.
5. Select Result as the target variable.
6. Split the dataset into 80% training data and 20% testing data.
7. Display the sizes of the training and testing datasets.
8. Create a KNN classifier with n_neighbors=3.
9. Perform 5-fold cross-validation using cross_val_score().
10. Display the accuracy scores obtained from the five folds.
11. Calculate and display the average accuracy.
12. Stop.



Result

The dataset was successfully split into training and testing sets, and the KNN classifier was evaluated using 5-fold cross-validation.

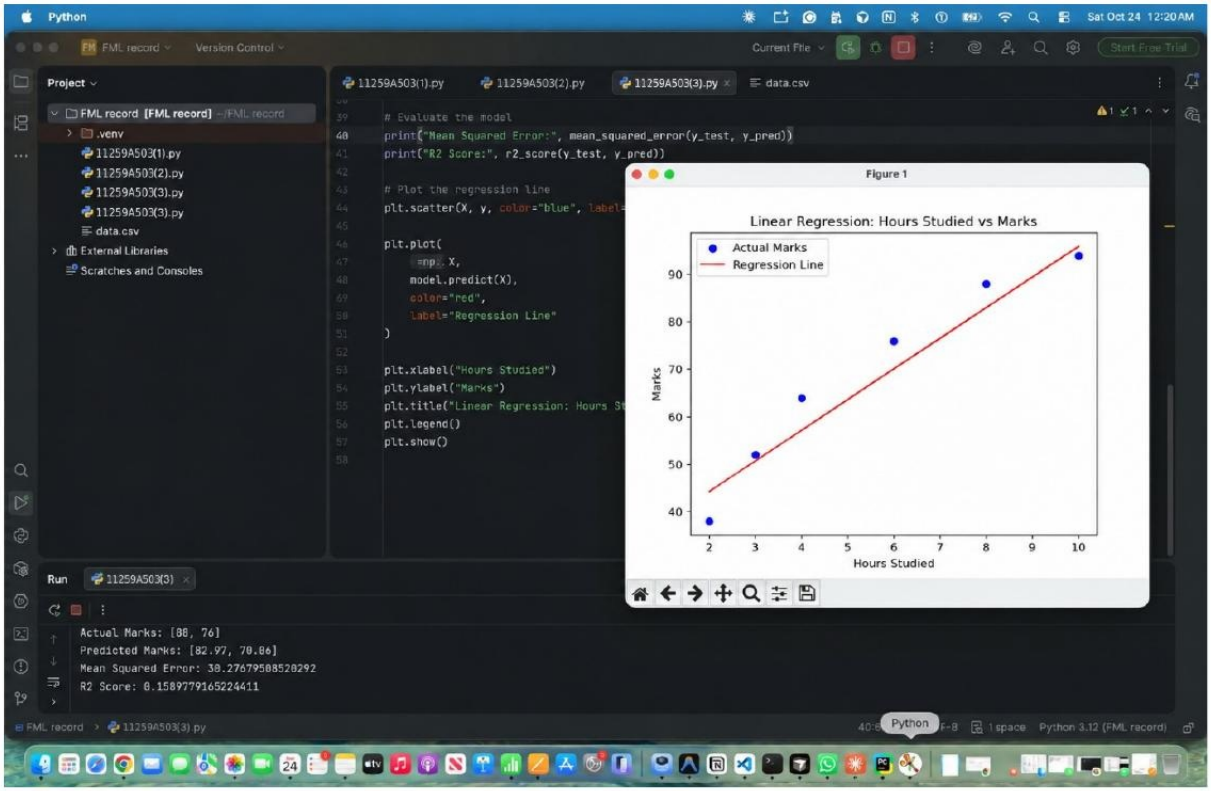
Ex.No. 3: Implement Linear Regression

Aim

To implement a Linear Regression algorithm to predict students' marks based on the number of hours studied and evaluate the model using Mean Squared Error (MSE) and R^2 score.

Algorithm

1. Start.
2. Import the required libraries: Pandas, Matplotlib, train_test_split, LinearRegression, and evaluation metrics.
3. Load the dataset from data.csv.
4. Select Hours_Studied as the input feature and Marks as the target variable.
5. Split the dataset into 80% training data and 20% testing data.
6. Create a Linear Regression model.
7. Train the model using the training dataset.
8. Predict students' marks using the testing dataset.
9. Evaluate the model using Mean Squared Error and R^2 score.
10. Display the predicted marks.
11. Plot the actual data points and the regression line.
12. Stop.



Result

Thus, the Linear Regression algorithm was implemented successfully to predict students' marks based on their study hours. The model's performance was evaluated using Mean Squared Error and R^2 score, and the actual data and regression line were plotted.

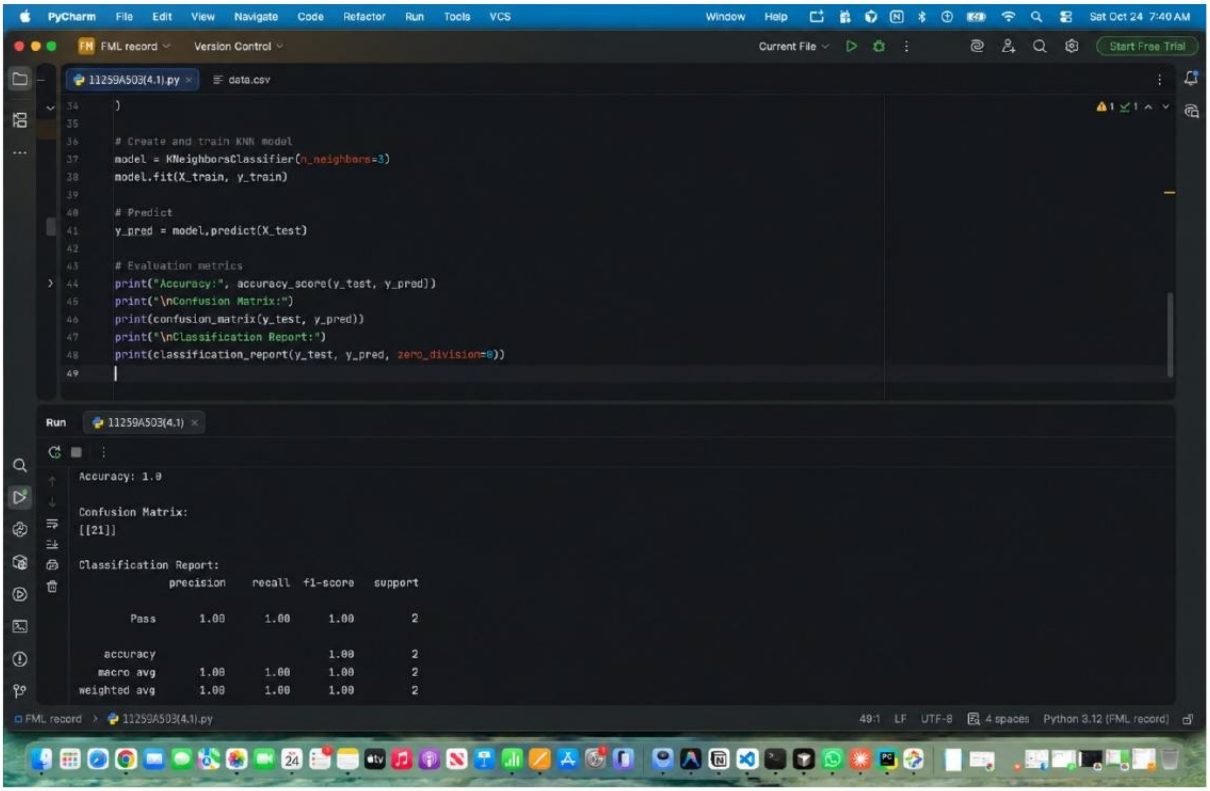
EX.NO. 4(A): Implement Classification Model Using KNN and Compute Evaluation Metrics

Aim

To implement the K-Nearest Neighbors (KNN) classification algorithm to predict students' results based on hours studied, attendance, and assignments, and evaluate the model using accuracy, confusion matrix, and classification report.

Algorithm

1. Start.
2. Import the required libraries: Pandas, train_test_split, KNeighborsClassifier, and evaluation metrics.
3. Load the dataset from data.csv.
4. Select Hours_Studied, Attendance, and Assignments as input features.
5. Select Result as the target variable.
6. Split the dataset into 80% training data and 20% testing data.
7. Create a KNN classifier with n_neighbors=3.
8. Train the model using the training dataset.
9. Predict the results for the testing dataset.
10. Calculate and display the accuracy score.
11. Display the confusion matrix.
12. Display the classification report containing precision, recall, and F1-score.
13. Stop.



Result

Thus, the KNN classification model was implemented successfully, and its performance was evaluated using accuracy, confusion matrix, precision, recall, and F1-score.

EX.NO. 4(B): Implement Classification Model Using SVM and Compute Evaluation Metrics

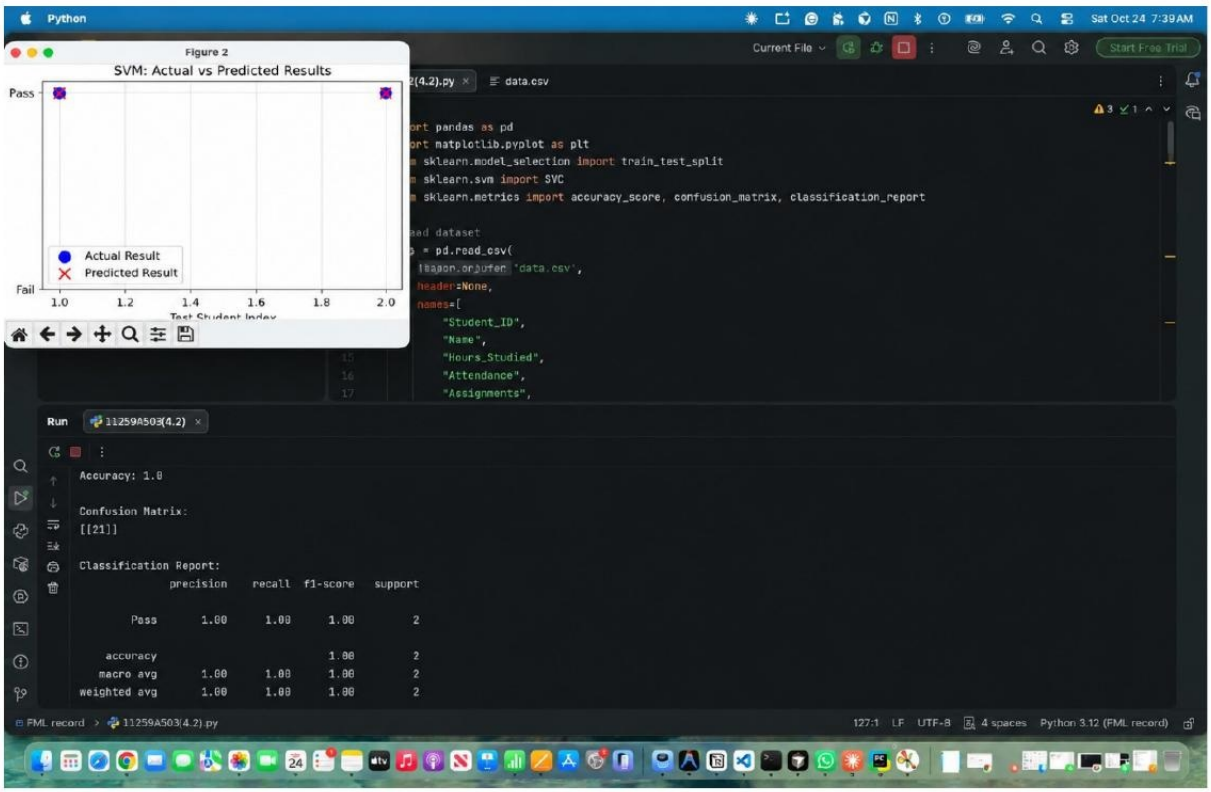
Aim

To implement a Support Vector Machine (SVM) classification algorithm using a linear kernel to predict students' results based on hours studied, attendance, and assignments, and evaluate its performance using accuracy, confusion matrix, and classification report.

Algorithm

1. Start.
2. Import the required libraries: Pandas, train_test_split, SVC, and evaluation metrics.
3. Load the dataset from data.csv.
4. Select Hours_Studied, Attendance, and Assignments as input features.
5. Select Result as the target variable.

6. Split the dataset into 80% training data and 20% testing data.
7. Create an SVM classifier using SVC(kernel='linear').
8. Train the model using the training dataset.
9. Predict the results for the testing dataset.
10. Calculate and display the accuracy score.
11. Display the confusion matrix.
12. Display the classification report containing precision, recall, and F1-score.
13. Stop.



Result

Thus, the SVM classification model with a linear kernel was implemented successfully, and its performance was evaluated using accuracy, confusion matrix, precision, recall, and F1-score.

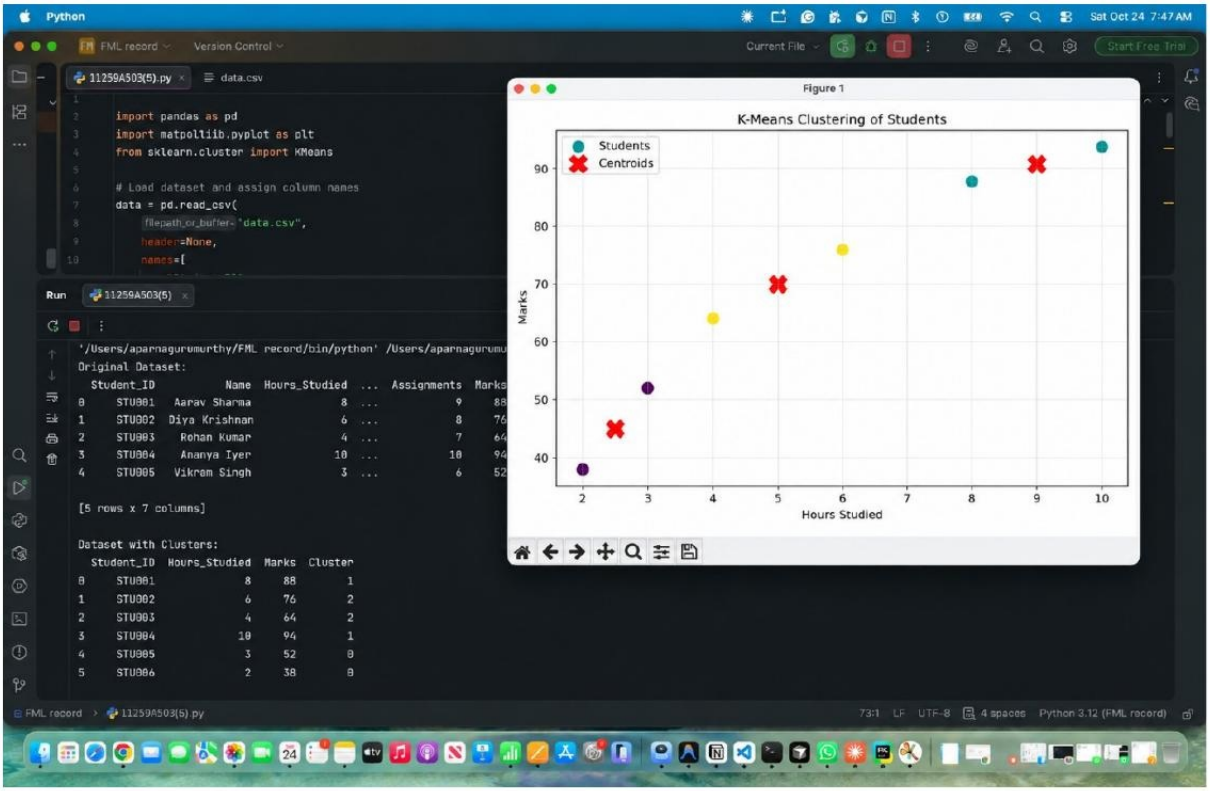
Ex No. 5: Implement K-Means Clustering

Aim

To implement the K-Means clustering algorithm to group students based on their hours studied and marks into three clusters and visualize the clusters along with their centroids.

Algorithm

1. Start.
2. Import the required libraries: Pandas, Matplotlib, and KMeans from sklearn.cluster.
3. Load the dataset from data.csv.
4. Select Hours_Studied and Marks as input features.
5. Create a K-Means clustering model with three clusters (n_clusters=3).
6. Fit the model to the selected data.
7. Obtain the cluster labels for each data point.
8. Add the cluster labels as a new column named Cluster in the dataset.
9. Display the clustered dataset.
10. Plot the data points using different colors for each cluster.
11. Plot the cluster centroids using an X-shaped marker.
12. Add axis labels, a title, and a legend to the graph.
13. Display the graph.
14. Stop



Result

Thus, the K-Means clustering algorithm was implemented successfully to group students into three clusters based on their hours studied and marks. The clustered data and centroids were visualized using a scatter plot.

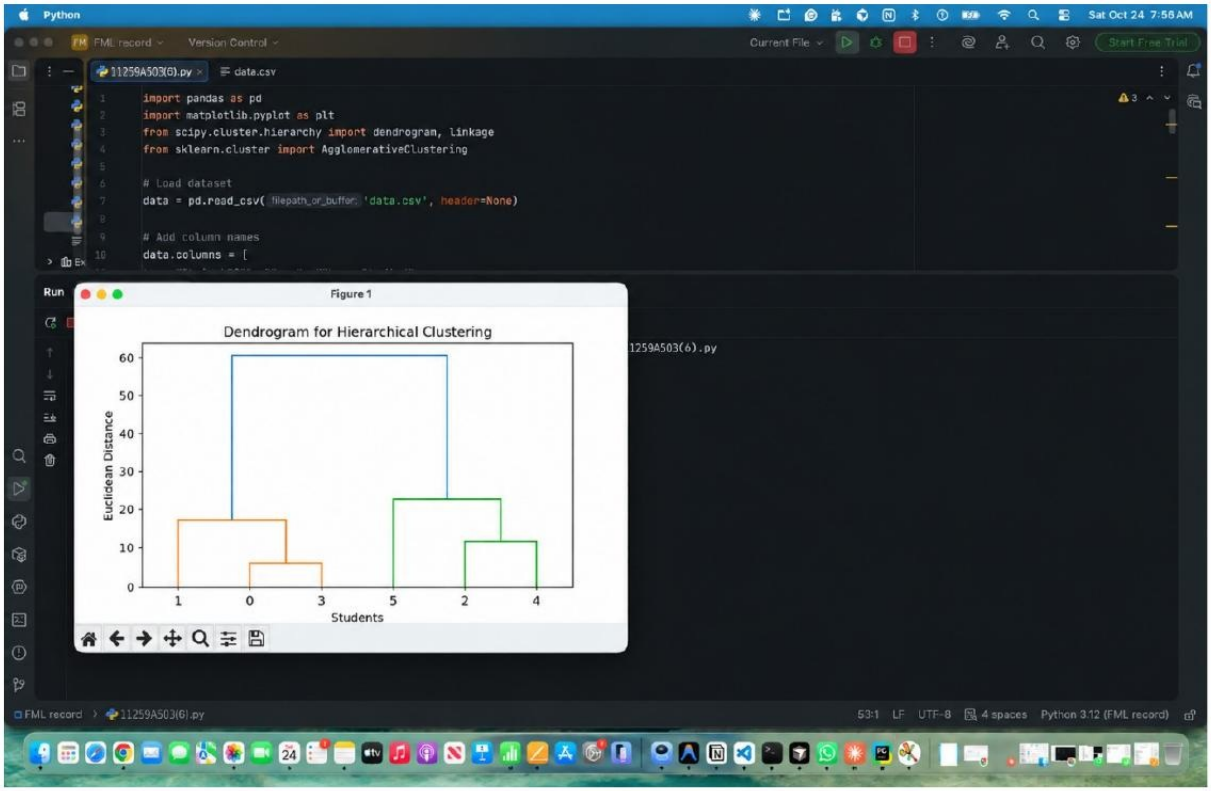
Experiment 6: Implement Hierarchical Clustering

Aim

To implement the Hierarchical Clustering algorithm to group students based on their hours studied and marks into three clusters, and visualize the results using a dendrogram and a scatter plot.

Algorithm

1. Start.
2. Import the required libraries: Pandas, Matplotlib, dendrogram, linkage, and AgglomerativeClustering.
3. Load the dataset from data.csv.
4. Select Hours_Studied and Marks as input features.
5. Create a linkage matrix using the Ward linkage method.
6. Plot the dendrogram to visualize the hierarchical relationships between data points.
7. Create an Agglomerative Clustering model with three clusters.
8. Apply the model to the selected data and predict cluster labels.
9. Add the cluster labels to the dataset as a new column named Cluster.
10. Display the clustered dataset.
11. Plot a scatter plot showing the three clusters using different colors.
12. Display the graph.
13. Stop.



Result

Thus, the Hierarchical Clustering algorithm was implemented successfully using Ward linkage and Agglomerative Clustering. The hierarchical relationships were visualized using a dendrogram, and the final three clusters were displayed using a scatter plot.

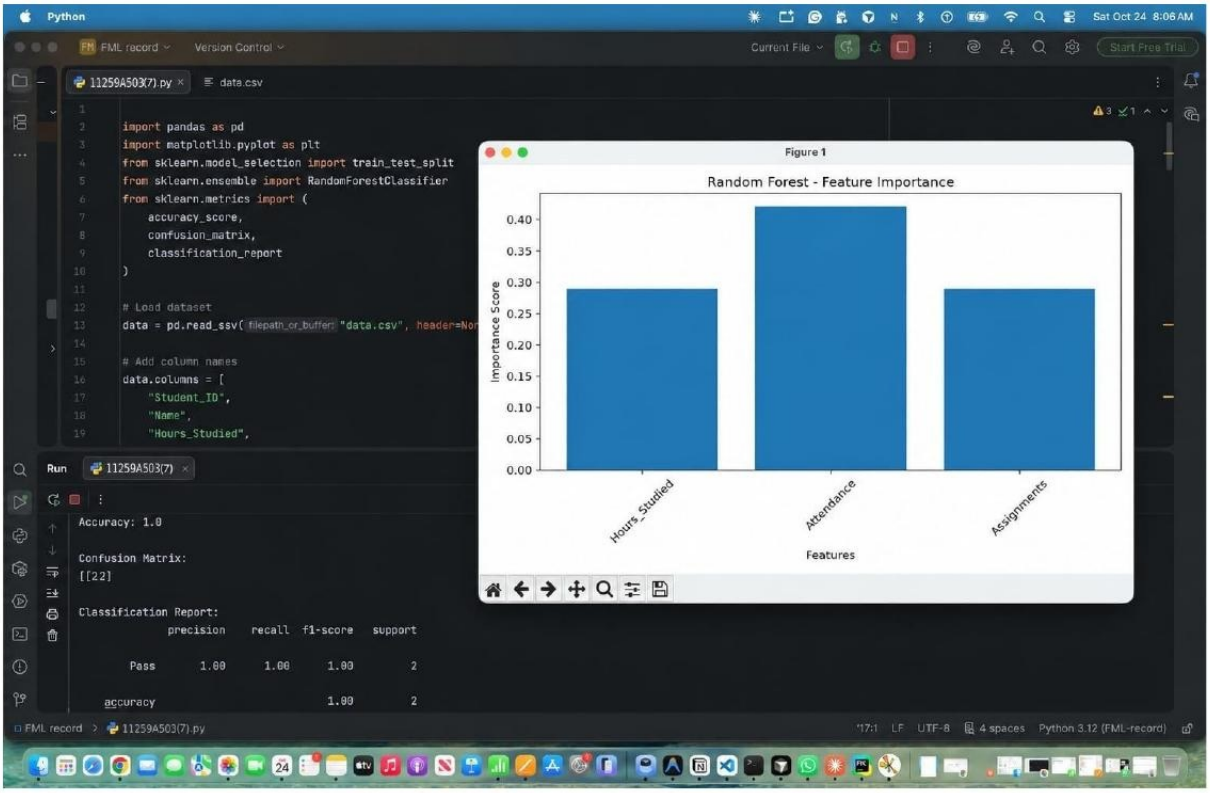
Experiment 7: Implement Random Forest and Analyze Performance

Aim

To implement the Random Forest classification algorithm to predict students' results based on hours studied, attendance, and assignments, and analyze its performance using accuracy, confusion matrix, classification report, and feature importance visualization.

Algorithm

1. Start.
2. Import the required libraries: Pandas, train_test_split, RandomForestClassifier, evaluation metrics, and Matplotlib.
3. Load the dataset from data.csv.
4. Select Hours_Studied, Attendance, and Assignments as input features.
5. Select Result as the target variable.
6. Split the dataset into 80% training data and 20% testing data using train_test_split().
7. Create a Random Forest classifier with 100 decision trees and random_state=42.
8. Train the model using the training dataset.
9. Predict the results for the testing dataset.
10. Calculate and display the accuracy score.
11. Display the confusion matrix and classification report.
12. Extract the feature importance values from the trained model.
13. Plot a bar graph showing the importance of each feature.
14. Display the graph.
15. Stop.



Result

Thus, the Random Forest classification algorithm was implemented successfully to predict students' results. Its performance was analyzed using accuracy, confusion matrix, precision, recall, and F1-score. Feature importance was also visualized using a bar graph.

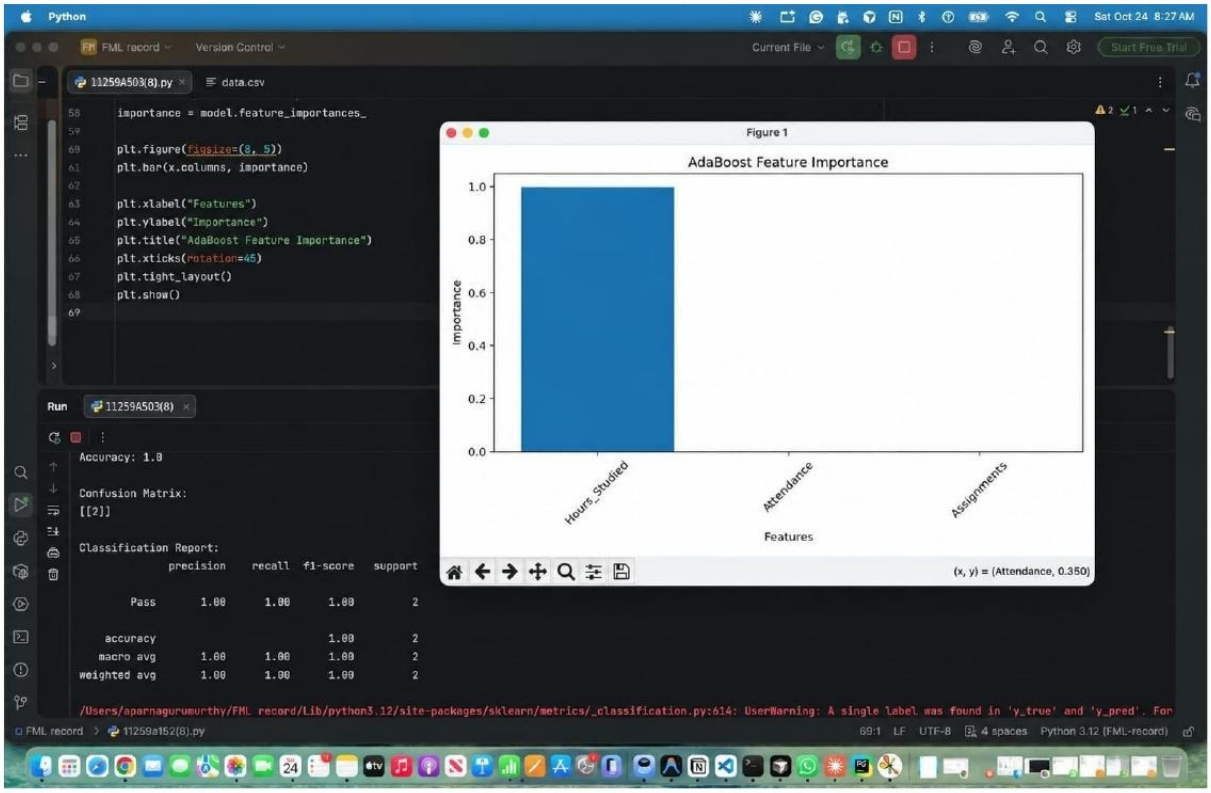
Experiment 8: Implement Boosting Technique (AdaBoost) and Analyze Performance

Aim

To implement the AdaBoost classification algorithm to predict students' results based on hours studied, attendance, and assignments, and analyze its performance using accuracy, confusion matrix, classification report, and feature importance visualization.

Algorithm

1. Start.
2. Import the required libraries: Pandas, Matplotlib, train_test_split, AdaBoostClassifier, and evaluation metrics.
3. Load the dataset from data.csv.
4. Select Hours_Studied, Attendance, and Assignments as input features.
5. Select Result as the target variable.
6. Split the dataset into 80% training data and 20% testing data.
7. Create an AdaBoost classifier with 50 estimators.
8. Train the model using the training dataset.
9. Predict the results for the testing dataset.
10. Calculate and display the accuracy score.
11. Display the confusion matrix and classification report.
12. Extract the feature importance values.
13. Plot a bar graph showing the importance of each feature.
14. Display the graph.
15. Stop



Result

Thus, the AdaBoost classification algorithm was implemented successfully. Its performance was analyzed using accuracy, confusion matrix, precision, recall, and F1-score, and feature importance was visualized using a bar graph.

Experiment 9: Implement a Basic Reinforcement Learning Model (Q-Learning)

Aim

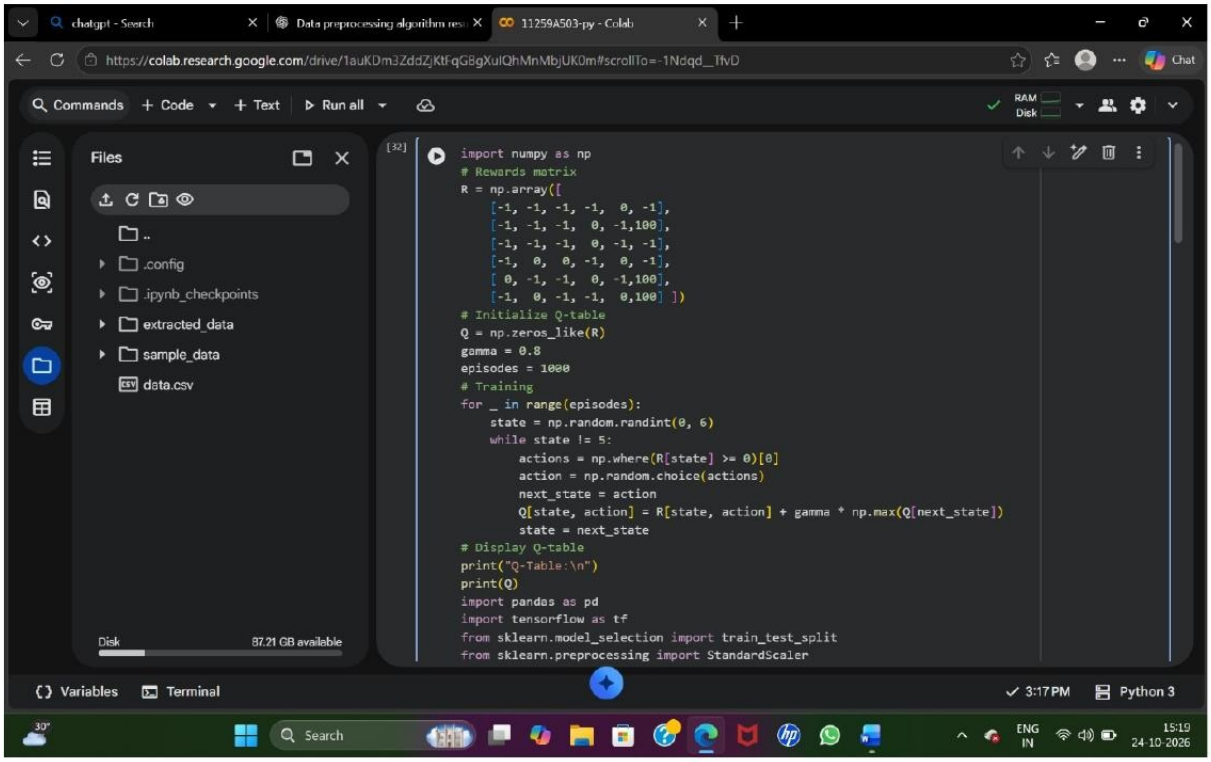
To implement a basic reinforcement learning model using the Q-Learning algorithm to learn the optimal actions for reaching a target state using a reward matrix and a Q-table.

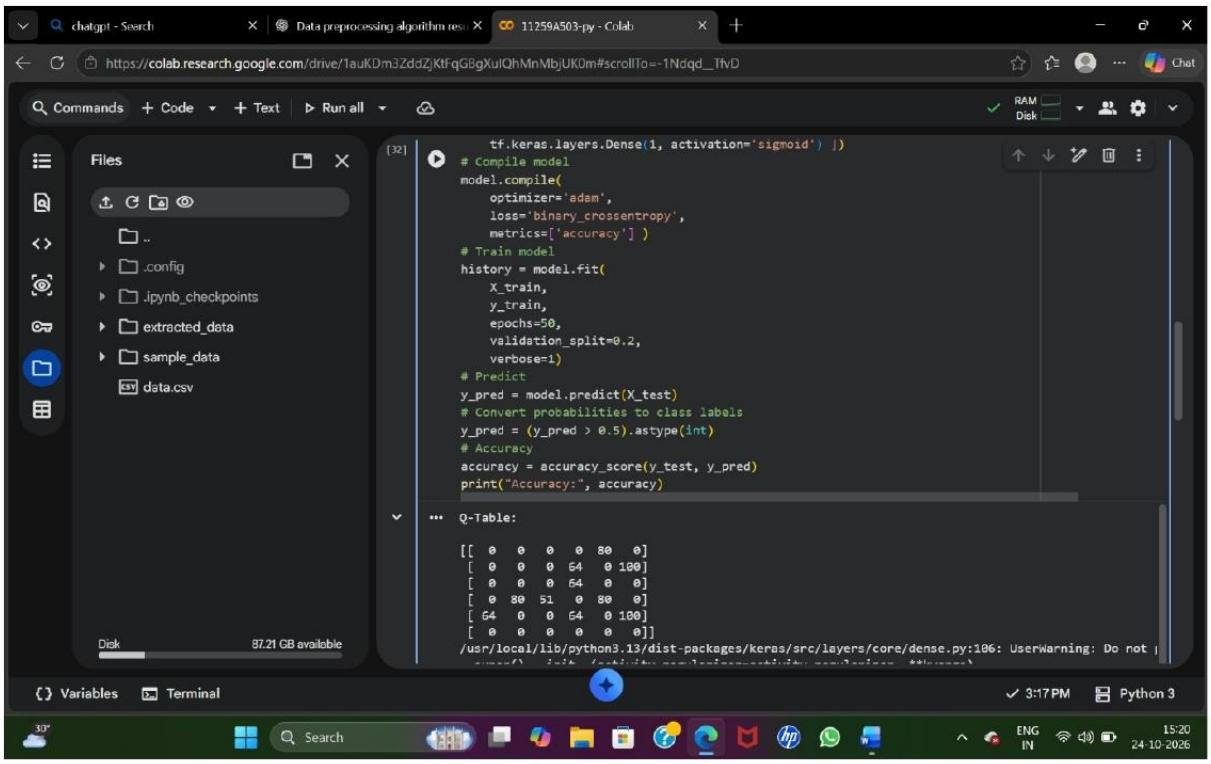
Algorithm

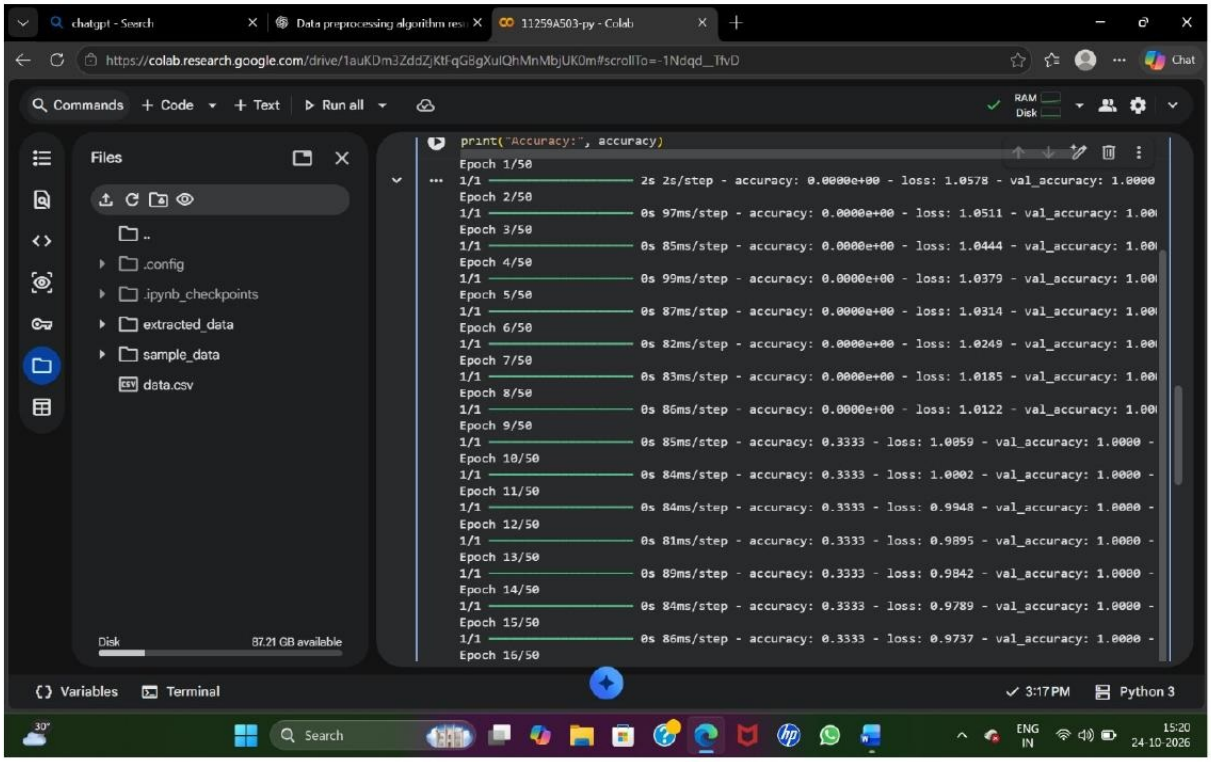
1. Start.
2. Import the NumPy library.
3. Define the reward matrix representing the rewards for moving between states.
4. Initialize the Q-table with zeros.
5. Set the discount factor \(\gamma = 0.8\) and the number of training episodes to 1000.
6. Randomly select an initial state.
7. Identify the possible actions from the current state using the reward matrix.
8. Randomly select one of the available actions.
9. Determine the next state based on the selected action.
10. Update the Q-table using the Q-Learning update equation:

\[Q(s,a)=R(s,a)+\gamma\max_{a'}Q(s',a') \]

11. Continue until the target state (state 5) is reached.
12. Repeat the training process for 1000 episodes.
13. Display the final Q-table.
14. Stop


chatgpt - Search
Data preprocessing algorithm resi
11259A503-py - Colab
https://colab.research.google.com/drive/1auKDm3ZddZjKtFqGBgXulQhMnMbjUK0m#scrollTo=-1Ndqd__TfvD
Commands
+ Code
+ Text
Run all
RAM
Disk
Files
..
.config
.ipynb_checkpoints
extracted_data
sample_data
data.csv
Disk
87.21 GB available
[32]
import numpy as np
Rewards matrix
R = np.array([
[-1, -1, -1, -1, 0, -1],
[-1, -1, -1, 0, -1,100],
[-1, -1, -1, 0, -1, -1],
[-1, 0, 0, -1, 0, -1],
[0, -1, -1, 0, -1,100],
[-1, 0, -1, -1, 0,100]])
Initialize Q-table
Q = np.zeros_like(R)
gamma = 0.8
episodes = 1000
Training
for _ in range(episodes):
state = np.random.randint(0, 6)
while state != 5:
actions = np.where(R[state] >= 0)[0]
action = np.random.choice(actions)
next_state = action
Q[state, action] = R[state, action] + gamma * np.max(Q[next_state])
state = next_state
Display Q-table
print("Q-Table:\n")
print(Q)
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
Variables
Terminal
3:17 PM
Python 3
Search
ENG
IN
15:19
24-10-2026


chatgpt - Search
Data preprocessing algorithm resi
11259A503-py - Colab
https://colab.research.google.com/drive/1auKDm3ZddZjKtFqGBgXulQhMnMbjUK0m#scrollTo=-1Ndqd__TfvD
Commands
+ Code
+ Text
Run all
RAM
Disk
Files
..
.config
.ipynb_checkpoints
extracted_data
sample_data
data.csv
Disk
87.21 GB available
[32]
tf.keras.layers.Dense(1, activation='sigmoid')])
Compile model
model.compile(
optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy'])
Train model
history = model.fit(
X_train,
y_train,
epochs=50,
validation_split=0.2,
verbose=1)
Predict
y_pred = model.predict(X_test)
Convert probabilities to class labels
y_pred = (y_pred > 0.5).astype(int)
Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
Q-Table:
[[0 0 0 0 80 0]
[0 0 0 64 0 100]
[0 0 0 64 0 0]
[0 80 51 0 80 0]
[64 0 0 64 0 100]
[0 0 0 0 0 0]]
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not
Variables
Terminal
3:17 PM
Python 3
Search
ENG
IN
15:20
24-10-2026



Result

Thus, the Q-Learning reinforcement learning model was implemented successfully. The agent learned state-action values through repeated interaction with the reward matrix, and the learned values were displayed in the Q-table.